

Assignment 1

Priyank

GV 271 - Graphics and Visualization, Indian Institute of Science • September 7, 2026

GPU: Mesa Intel(R) Iris(R) Xe Graphics (TGL GT2)

CPU: 11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz

This assignment renders a protein molecule whose structure is defined by the provided OFF files. The mesh rotates continuously about a randomly chosen axis with a fixed angular speed.

1 Task 1: Model Loading, Camera Setup, Rotation, and Base Rendering

(i) **Approach.** The OFF parser reads vertices/polygon indices into a flat indexed triangle buffer. Since OFF carries no normals, per vertex normals are derived from the face normals of the incident triangles, under either of two weighting schemes selectable at runtime (area or angle; see (iii)). The mesh rotates continuously about an axis drawn uniformly at random once per run (`initRandomAxis`), advancing every frame at a fixed rate driven by wall clock time (`timer`), composed into the model matrix with the centering translation and normalizing scale.

(ii) **Core files/functions.** `main.cpp`: `readOffFile`, `createVertexBuffers`, `createVertexNormals`, `ComputeModelTransform`, `setupCamera`, `updateProjection`, `initRandomAxis`, `timer`. `shader.vs/shader.fs` (`shaderProgram`) render the whole mesh.

(iii) **Challenges / design decisions.** Two vertex normal weighting schemes are implemented and switchable at runtime (`n`, via `currentNormalMode/rebuildNormals`). Accumulating *unnormalized* cross products gives area weighting for free, since a cross product's magnitude is twice the triangle area; the default instead normalizes each face normal and weights it by the interior angle at the vertex, which is insensitive to triangulation density, so a dense patch of small triangles cannot outvote a coarser neighbouring region sharing the same vertex. Face normals are retained separately (`faceNormals`) and drive a flat shading mode for comparison. A random rotation axis can, by chance, land close to the camera's fixed view direction, under which rotation produces almost no visible depth change and the object appears to spin flat rather than tumble in 3D; candidate axes within 32° of the view direction are rejected and redrawn.

2 Task 2: Depth Based Colormap Shading

(i) **Approach.** Lighting uses (ambient + diffuse + Phong(R,V)). The diffuse base color comes from a depth driven colormap (viridis or coolwarm) instead of a flat material, so once the mesh is cut open in Tasks 3 and 4, interior faces are distinguished by depth without needing a second material.

(ii) **Core files/functions.** `shader.fs`: `colormapViridis`, `colormapCoolWarm`, lighting in `main()`. `shader.vs`: `vDepth01` from `-viewPos.z`. Uniforms `uKa/uKd/uKs/uShininess`, `uNearDepth/uFarDepth` set once, Functions : `faceNormals`, `rebuildNormals()`, `buildFlatShadedMesh()`, in `main.cpp`.

(iii) **Challenges / design decisions.** Lighting was evaluated per fragment in shaders rather than on the CPU specifically for performance: every quantity it depends on (rotated normal, view vector, depth color) changes every frame under continuous rotation regardless of implementation, and the GPU evaluates this in parallel across thousands of cores, avoiding both a serial per pixel CPU loop and again uploading a full color buffer every frame. Depth uses view-space `-viewPos.z` (not NDC z) so the ramp stays linear regardless of projection type. Colormaps are piecewise-linear keyframe tables, trading exact colour fidelity for shader simplicity. A `uUseVertexColor` toggle isolates shading bugs from geometry bugs during debugging.

3 Task 3: CPU Side Slicing

(i) **Approach.** M evenly spaced lines through the origin ($\theta_i = i\pi/M$) give $2M$ rays around the full circle; each wedge is bounded by two half planes from its boundary rays rotated $\pm 90^\circ$. Every triangle is clipped against both half planes in sequence (a Sutherland Hodgman clip specialized to triangles), and surviving geometry per wedge is pushed outward along its bisector for the exploded view; all buffers rebuild every frame. At $M = 1$ the two wedges are further distinguished by polygon mode: one `GL_FILL`, one `GL_LINE`: satisfying the base solid/wireframe requirement; at $M > 1$ all wedges are filled.

(ii) **Core files/functions.** `main.cpp`: `rebuildWedgeGeometryDefs` (angles, half plane normals, bisectors), `clipListAgainstPlane` (lone vertex detection, edge intersection, 1 or 2 triangle output), `lerpVert`, `buildAllWedgesCPU`, `uploadSliceVAO`. The $M = 1$ solid/wireframe pair is applied in `display` (the `wireframeThis` branch around `glPolygonMode`).

(iii) **Challenges / design decisions.** The "lone" vertex is found without sorting: with three booleans and two possible values, at least two must agree, so a pigeonhole case split locates the odd one out in $O(1)$. The intersection parameter $t = d_a/(d_a - d_b)$ is guaranteed in $(0,1)$ once the lone vertex's sign is confirmed opposite its neighbors. Indices are cyclically relabeled ($a, b, c = \text{lone}, (\text{lone}+1)\%3, (\text{lone}+2)\%3$), preserving winding so lighting/culling stay correct post clip. The cost is performance: the full mesh is reclipped and every wedge's VBO/EBO reuploaded every frame, motivating Task 4.

The clipper is parameterized by signed distance $d = \mathbf{n} \cdot \mathbf{p} - w$ rather than assuming planes through the origin, so plane position is a runtime scalar. Setting $w \neq 0$ (o, via `wedgeCutOffset`) slides each wedge boundary off the scene centre in both the CPU and GPU paths. Note that wedges presuppose concurrency: with the boundaries moved apart, adjacent wedges no longer share a cut line, so small gaps open between them: a property of the wedge decomposition, not of the clip.

4 Task 4: GPU-Side Slicing via Geometry Shader

(i) **Approach.** The same clip algorithm runs as a per triangle geometry shader stage in local space, reusing one static buffer every frame with no CPU rebuild. A pass through vertex shader forwards local position/normal; the geometry shader clips against `uWedgeNormalA`, then `uWedgeNormalB` (two chained half space clips = the wedge), applying MVP and the offset only to emitted vertices. The Task 3 fill/wireframe rule at $M = 1$ is mirrored here via `glPolygonMode` around the GPU draws.

(ii) **Core files/functions.** `slice.vs` (pass through). `slice.gs`: `clipTri`, `main()` (two stage clip chaining), `emitWorld/emitTri`. `shader.fs` reused unchanged. `main.cpp`: `buildPreppedWholeMesh`, uniform wiring for `sliceGSProgram`.

(iii) **Challenges / design decisions.** `max_vertices=12` covers the worst case: the first clip yields at most 2 triangles, each of which the second clip splits into at most 2 more, giving 4 triangles of 3 vertices. Each is emitted as an independent primitive (`EndPrimitive()` after every third vertex), so the declared `triangle_strip` output carries no shared vertices. Normals are renormalized after interpolation, since the linear blend of two unit vectors is not itself unit length.

The key bug was variable shadowing: locally redeclaring `uWedgeNormalA/BLoc` during init left the globals at their zero initialized value. Since 0 is itself a valid uniform location, the wedge normals were silently written into whichever uniform occupied slot 0 rather than failing loudly, and the clip planes stayed at (0,0,0) - for which $\text{dot}(p, n) > 0$ is false everywhere, discarding every triangle. Relatedly, uniform storage is per-program, not per name: lighting uniforms set only on `shaderProgram` left the GPU sliced mesh pure black until set again on `sliceGSProgram`.

When slices overlap, the extra cost lands on the fragment stage rather than the geometry stage. If d is small, neighbouring wedges still cover some of the same pixels, and those pixels get shaded more than once: work that is thrown away when a later fragment overwrites an earlier one. Both implementations pay the same absolute cost here, since both rasterize the same overlapping geometry through the same fragment shader. What differs is how much it matters. A CPU frame is dominated by `buildAllWedgesCPU` reclipping and reuploading the mesh - over three seconds per frame on 20AR_1- so a sub millisecond overdraw penalty disappears into the noise. GPU frames run in a few milliseconds, so the same penalty is a visible fraction of the total.

5 Performance Analysis: CPU vs. GPU Wedge Slicing

(i) **Approach.** FPS is measured by accumulating frames over fixed wall-clock windows (`glutGet(GLUT_ELAPSED_TIME)` in `timer`), shown in the HUD. Two axes are varied: active planes M (1 to 10, via j/k, calling `rebuildWedgeGeometryDefs`; sampled at $M \in \{1, 4, 10\}$ for the table) and mesh resolution (separate OFF files). Each (resolution, M) pair is logged for both CPU and GPU modes, with non parallel plane orientations.

(ii) **Core files/functions.** `main.cpp`: `timer`, `keyboard` (j/k), `display` (mode dispatch).

(iii) **Expected behavior, reasoning, and results.** CPU cost scales with (triangle count $\times 2M$ wedges), since `buildAllWedgesCPU` again clips and again uploads every wedge every frame. GPU cost grows far more gently with M : each wedge costs one more draw call plus inherently parallel `slice.gs` work, with no reclip- `buildPreppedWholeMesh`'s buffer uploads once, regardless of M . Higher resolution scales the CPU loop directly and serially, while the GPU's per primitive invocation is parallel and degrades more slowly until cores saturate. Wedge boundaries crossing the mesh's dense interior force more triangles through the expensive split and interpolate branch on both implementations- cheap on the GPU (a few extra `EmitVertex` calls in GPU memory), costlier on the CPU (more interpolation and a bigger reupload).

Mesh	Vertices	Triangles	Planes (M)	CPU FPS	GPU FPS
1GRM_1	83673	167346	1	5.5	283.9
1GRM_1	83673	167346	4	3.8	152.4
1GRM_1	83673	167346	10	2.4	68.4
1GRM_2	41836	83672	1	11.3	509.3
1GRM_2	41836	83672	4	7.6	255.3
1GRM_2	41836	83672	10	4.7	121.6
1GRM_4	20918	41836	1	21.9	776.0
1GRM_4	20918	41836	4	14.8	431.4
1GRM_4	20918	41836	10	9.0	228.6
1GRM_8	10459	20918	1	41.6	1000.0
1GRM_8	10459	20918	4	29.0	715.7
1GRM_8	10459	20918	10	16.5	358.1
1GRM_16	5229	10458	1	82.9	1000.0
1GRM_16	5229	10458	4	52.6	978.6
1GRM_16	5229	10458	10	32.7	516.4
20AR_1	519221	1038654	1	0.8	74.8
20AR_1	519221	1038654	4	0.6	35.2
20AR_1	519221	1038654	10	0.4	15.0
20AR_2	259557	519326	1	1.7	125.5
20AR_2	259557	519326	4	1.2	63.7
20AR_2	259557	519326	10	0.7	28.1
20AR_4	129725	259662	1	3.5	209.7
20AR_4	129725	259662	4	2.4	120.2
20AR_4	129725	259662	10	1.4	54.7
20AR_8	64810	129830	1	6.9	362.3
20AR_8	64810	129830	4	4.8	205.7
20AR_8	64810	129830	10	2.9	97.7
20AR_16	32361	64914	1	13.8	600.0
20AR_16	32361	64914	4	9.5	364.3
20AR_16	32361	64914	10	5.7	161.2
3SY7_1	221469	442946	1	2.0	141.4
3SY7_1	221469	442946	4	1.4	75.0
3SY7_1	221469	442946	10	0.9	32.9
3SY7_2	110733	221472	1	4.1	253.9
3SY7_2	110733	221472	4	2.9	131.9
3SY7_2	110733	221472	10	1.8	60.7
3SY7_4	55365	110736	1	6.6	431.4
3SY7_4	55365	110736	4	5.7	236.5
3SY7_4	55365	110736	10	3.5	107.0
3SY7_8	27680	55368	1	16.3	688.6
3SY7_8	27680	55368	4	11.0	402.3
3SY7_8	27680	55368	10	6.8	179.5
3SY7_16	13840	27684	1	31.7	958.6
3SY7_16	13840	27684	4	21.5	610.0
3SY7_16	13840	27684	10	13.1	298.1

Table 1: Measured FPS across mesh resolutions and active cutting planes, CPU vs. GPU wedge slicing.

Extension: Non Concurrent Cutting Planes

Tasks 3 and 4 cut with planes that all pass through the scene centre. That restriction is what keeps the decomposition simple: M concurrent planes always give exactly $2M$ wedges, each bounded by two half-planes, so the regions can be worked out in advance (`rebuildWedgeGeometryDefs`) and each wedge costs the same two clips no matter how large M is.

Mode 4 (4) removes that restriction. With planes in general position, the number of regions depends on where the planes happen to fall, so it cannot be computed up front. `buildArrangementCPU` therefore finds the regions by cutting repeatedly: start with the whole mesh, and for each plane split every surviving piece in two, keeping both halves and discarding anything empty.

Each split also pushes its two halves apart along the normal of the plane that separated them, and these displacements add up as a piece passes through successive planes. This is what guarantees separation: two pieces divided by a plane always move in opposite directions along it, whatever their size. Pushing each piece along its own centroid direction does not work, because a thin sliver and the slab next to it have almost the same centroid and end up on top of each other.

None of this ports to the geometry shader. A GS handles one triangle at a time and carries no state between passes, so the accumulated offsets have nowhere to live, and M chained clips would need $3 \cdot 2^M$ output vertices against a fixed `max_vertices` budget.

Keyboard Controls

Key	Action
1/2/3/4	Whole mesh / CPU slice / GPU slice / non-concurrent planes
j/k	Decrease / increase active cutting planes (M)
o	Toggle cutting planes through-centre / off-centre
n	Toggle vertex-normal weighting (area / angle)
m	Toggle flat (face-normal) / smooth (vertex-normal) shading
v	Cycle colormap (viridis / coolwarm)
c	Toggle lit colormap vs. flat colour
f	Toggle wireframe / filled
d	Toggle dark / light background
+/-	Zoom in / out
b	Run the benchmark sweep